does mac provide file events

Yes, macOS provides built-in file system event monitoring through FSEvents (File System Events API), which tracks creations, modifications, deletions, and renamings across directory trees. For fine-grained or real-time tracking, macOS also offers alternative system tools depending on your exact monitoring needs. Stack Overflow +3

Built-in File Event Technologies

- **FSEvents API (`fseventsd`):** The primary framework for tracking changes across directory hierarchies without constant disk polling. It reports when files or folders are created, modified, deleted, or moved, and powers features like Spotlight and Time Machine. Note that standard FSEvents track changes at the directory level rather than individual read/open operations. HackMD +4
- **`kqueue` / `kevent`:** A system call mechanism used by applications to monitor specific, individual file or directory descriptors for changes like writes, truncation, or deletion. Stack Overflow
- **`fs_usage` (Command Line):** A terminal utility that prints live system calls, allowing you to trace all file system activity and access patterns in real time (similar to `strace` on Linux). You can inspect this using the Tracing MacOS Filesystem Events Guide.
- **Endpoint Security System Extensions:** For deep, security-grade monitoring (such as catching file open or read operations on modern macOS versions), developers use Apple's Endpoint Security framework rather than standard file events. Stack Overflow

Are you trying to ↪ **monitor file changes for a specific app/script**, or do you need to

Ask anything

File-level filesystem change notification in Mac OS X It tells you only that something in the directory has changed, but does not ...

Apple Developer

File System Events - Documentation - Apple Developer

Overview. The file system events API provides a way for your application to ask for notification when the contents of a directory ...

Stack Overflow

macos - How to monitor file open and file read events in OSX

File Event Monitoring on macOS: Monitoring file open and file read events on macOS is difficult, as standard file system events on...

Show all

i want a background damon that would monitor specific folders for file changes

To create a background daemon that monitors specific folders for file changes on macOS, the native, production-grade approach is to use a **`launchd` daemon paired with `WatchPaths`**.

This method requires zero programming or external tools—it relies entirely on a standard configuration file (`.plist`) that triggers a script or command whenever a specified folder changes.

Step-by-Step Implementation

1. Write the Monitoring Script

Create the script or command you want to run when a file change happens. For this example, let's create a simple Bash script that logs changes.

Use code with caution.

2. Paste the following logic:

```bash
#!/bin/bash
echo "Change detected in monitored folder at $(date)" >> ~/Documents/
```

Use code with caution.

3. Save, exit (`Ctrl+O` , `Enter` , `Ctrl+X`), and make the script executable:

```bash
chmod +x ~/Documents/my_monitor_script.sh
```

Use code with caution.

2. Create the `launchd` Configuration File (.plist)

Mac daemons are controlled by Property List (`.plist`) files.

1. Create a new plist file in your user Agents directory:

```bash
nano ~/Library/LaunchAgents/com.user.foldermonitor.plist
```

Use code with caution.

2. Paste the following XML configuration, making sure to update the path to your target folder and your script:

```
    <key>Label</key>
    <string>com.user.foldermonitor</string>

    <key>ProgramArguments</key>
    <array>
        <string>/Users/YOUR_USERNAME/Documents/my_monitor_script.sh</
    </array>

    <key>WatchPaths</key>
    <array>
        <string>/Users/YOUR_USERNAME/Path/To/Monitored/Folder</string
    </array>
</dict>
</plist>
```

Use code with caution.

(Note: Replace `YOUR_USERNAME` and `/Users/YOUR_USERNAME/Path/To/Monitored/Folder` with your actual Mac paths. Tilde `~` expansion does not work inside plists, so you must use full absolute paths).

3. Load and Start the Daemon

To tell macOS to read your new configuration file and start monitoring, load it into the system's launch manager:

bash

```
launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/com.user.folderm
```

Use code with caution.

To test if it works, drop a file into your monitored folder. You should instantly see a new

If you are building an actual application (Python, Node.js, Go) rather than a system-level utility, do not write raw plists. Instead, use a library that abstracts macOS `FSEvents` directly into your code:

- **Python:** Use the `watchdog` library (`pip install watchdog`). It defaults to macOS `FSEvents` out of the box.
- **Node.js:** Use `chokidar` (`npm install chokidar`), which is highly optimized for macOS file monitoring.

Would you prefer to proceed with this **native `launchd` approach**, or are you writing code in a language like **Python or Node.js** where an application library makes more sense?



i am adding to a cli journaling tool i made in python

Since you are building a Python CLI tool, the best approach is to use the **`watchdog`** library. It uses native macOS `FSEvents` under the hood, meaning it is highly efficient, consumes virtually no CPU while idling, and doesn't require polling the disk.

Here is how to implement a background monitor directly inside your Python project.

1. Install Watchdog

Run this in your project environment:

bash

```
pip install watchdog
```







Add this background monitoring logic to your CLI code. This script sets up an observer that watches a specific folder and triggers an event handler whenever a file is created, modified, or deleted.

python

```python
import time
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class JournalFolderHandler(FileSystemEventHandler):
    """Handles file system events for your journal directory."""

    def on_modified(self, event):
        if not event.is_directory:
            print(f"📝 Journal file modified: {event.src_path}")
            # Add your custom logic here (e.g., auto-commit, sync, re-ir

    def on_created(self, event):
        if not event.is_directory:
            print(f"🆕 New journal entry detected: {event.src_path}")

    def on_deleted(self, event):
        if not event.is_directory:
            print(f"🗑️ Journal file deleted: {event.src_path}")

def start_monitoring(path_to_watch):
    event_handler = JournalFolderHandler()
    observer = Observer()

    # recursive=True will also monitor subfolders if your journal is nes
    observer.schedule(event_handler, path=path_to_watch, recursive=False
    observer.start()

    print(f"👀 Monitoring journal folder: {path_to_watch}")
    print("Press Ctrl+C to stop.")
```

```
    observer.join()

if __name__ == "__main__":
    # Replace with your actual journal directory path
    JOURNAL_DIR = "./my_journals"
    start_monitoring(JOURNAL_DIR)
```

Use code with caution.

3. Making it a True Background Daemon

Running the script above blocks your terminal. To make your CLI spin this up as a true background process (daemon) that runs silently even after the user closes the CLI, use Python's `subprocess` or `multiprocessing` to detach it, or use the `launchd` plist approach from earlier to run the script automatically when the Mac boots.

What action do you want your CLI to take when a journal file changes (e.g., **auto-committing to Git**, **syncing to a cloud database**, or **updating a local index**)?